Know which queries can leave Databricks before they do.

Nirvana runs every query on Databricks and on Zetaris against the same data, compares the answers row by row, and keeps a rewrite only when it returns the exact rows.

The problem

A migration fails on answers, not on syntax.

The push

Azure Databricks Standard workspaces are being moved to Premium from 1 October 2026. Zetaris puts the increase at 35% or more per workspace.

The risk

A query that runs on a new engine can still return a different answer: a DECIMAL read as DOUBLE, 7 / 2 = 3, NULLs sorted to the other end of a top 10.

The gap

Table diffs prove the data moved. Nobody proves, query by query, that the workload still gives the same answers, and which rewrite fixes the ones that do not.

How it works

Four steps. Every one executed.

01

Align the inputs

Both engines profile every table: row count, and per column non-null count, MIN, MAX and an exact SUM. A query over a table that differs is blocked, not judged.

02

Run on both

The same query runs on Databricks and on Zetaris Lightning, which reads the parquet files in place.

03

Compare every row

Results become canonical tokens with exact decimals and a SHA-256 fingerprint. A changed value is named by its column.

04

Prove the cause

Rewrites run on Zetaris until one returns the source's exact rows. Otherwise the query stays on Databricks.

The guarantee

No verdict comes from a model.

A model may only propose: the next check, a repair, a rewrite. The engines run it, and Nirvana keeps it only if the rows equal Databricks' rows.

A proposal with the right shape and the wrong rows is refused (test_wrong_advisor_rewrite_is_refused). An offline model ends a query in "keep", never in a guess.

Verdict	Meaning
move	same answer on both engines
move_note	same up to float noise or tied rows, proven by extra runs
rewrite	moves after a rewrite proven on the rows
keep	no rewrite reproduced the answer
blocked	input differs, or the source refuses the query

The live run · 27 Oct 2026, 18:29 UTC

102 of 103 TPC-DS queries can move.



Same answer **96** Same, with a note **6** Cannot be judged **1**

Tables aligned	Databricks time, all queries	Zetaris time, all queries	Run
24/24	247 s	272 s	run-20261027-181712-e4eb1f

Databricks Free Edition (serverless SQL warehouse) against Zetaris Lightning (local docker), the same 24 TPC-DS sf1 parquet files on both. Query texts: Apache Spark's TPC-DS set, unchanged.

Queries that did not simply match

Every exception carries its reason.

Query	Verdict	Why
q6	Same, with a note	same rows; the order differs only where the ORDER BY does not decide it
q30	Cannot be judged	the source cannot run this query: FAILED: [UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or ...
q56	Same, with a note	same rows; the order differs only where the ORDER BY does not decide it
q58	Same, with a note	floating-point differences only, largest relative gap 9.52e-11
q64	Same, with a note	same rows; the order differs only where the ORDER BY does not decide it
q75	Same, with a note	same rows; the order differs only where the ORDER BY does not decide it
q77	Same, with a note	same rows; the order differs only where the ORDER BY does not decide it

What alignment caught on day one

Zetaris' bundled TPC-DS sample is not Databricks' sample.

Table	Databricks samples	Zetaris TPCDS_DB	Gap
catalog_page	11,718	311,718	+300,000
catalog_returns	143,672	144,067	+395
catalog_sales	1,439,935	1,621,741	+181,806
store_returns	288,279	287,514	-765
store_sales	2,879,789	2,880,404	+615
web_returns	71,772	71,763	-9
web_sales	718,931	719,380	+449

7 of 24 tables differ in row count before a single query runs.

Comparing queries over these tables would have compared two datasets, not two engines. Alignment blocks them, so the live run loads the same parquet files into both engines instead.

The CSV-export path, live on Zetaris

A real migration path, and what Zetaris caught.

3 rewrites proven

Loading the same tables from CSV with inferred types turned DECIMAL into DOUBLE. q7, q26 and q28 returned different averages; casting the columns back on Zetaris reproduced Databricks' exact rows.

1,351 rows, found in one query

Zetaris read customer_address from Databricks itself (its bundled Databricks JDBC driver) and from the CSV copy at once. Every differing row is a ZIP code that lost its leading zero: 09082 became 9082.

Enforced by Zetaris

Nirvana measured each column on Databricks and installed it as Zetaris data-quality rules on the migrated table. The ZIP rule breaks on the same 1,351 rows, and keeps checking after every later load.

The agent

Nemotron picks the next check. The rows decide.

Blocked table	Steps	Time	Recalled	Proven on every row
customer_address	4	104 s	0	ca_street_number: unchanged, type only; ca_zip: repaired by lpad(cast(x as string), 5, '0')
store	2	35 s	2	s_street_number: unchanged, type only
web_site	2	70 s	3	web_street_number: unchanged, type only

Each step, Nemotron (through Nightshift) chooses one tool: profile a column, compare it row by row in one federated Zetaris query, verify a repair, install a guard, or try a rewrite.

A column is cleared only by 0 differing rows; a repair only by 0 mismatches across the whole table. Findings go to H-MEM and are recalled before the next case's first step. 2 query investigation(s) found no fix that matched the rows, so those queries stay on Databricks.

The overnight audit · 27 Oct 2026, 23:52 UTC

47 → 100 of 103 queries can move, unattended.

Unblocked by the agent	Blocked tables cleared	Guard rules installed	Took
53	3/3	2	33 min

One job: run all 103 queries, investigate everything blocked or kept, re-judge with what was proven, install Zetaris guard rules for every repair, write the morning report. 3 queries stay on Databricks, each with its reason; the agent could not fix q66.

Sponsor technology, load-bearing

Remove any one and a step stops.

Zetaris · the data layer

The 24 parquet files are Lightning filestore tables, read in place. Every query, table profile and rewrite runs through the REST SQL endpoint. It reads Databricks over JDBC in the same query as its own tables, and enforces the source's measured shape as DQ rules.

Meterless · H-MEM and Nightshift

H-MEM stores each proven fix and each order-tie proof with its query features. Matching queries try recalled fixes first and skip re-runs already settled; a fix that fails loses confidence. Nightshift is the only AI provider, on 127.0.0.1:8088, under the 0.90 provider contract.

NVIDIA Nemotron

Nightshift relays every agent step to a free Nemotron, which picks the next check and proposes repairs and rewrites. No local model runs. The engines decide on the rows.

Engineering

Built to be checked.

1,343 tests

Real SQL on DuckDB, the real H-MEM engine, every TPC-DS query parsed and qualified, tampered tables caught, property tests on the canonicaliser.

Receipts

Each run gets a SHA-256 over query hashes, both fingerprints, verdicts and rewrites. Importing an edited recording is refused.

A dashboard, not a slide

Start a run, watch 24 tables and 103 queries fill in live, open any query to its rows, fingerprints, rewrite and every execution step.

Honesty

What this is, and what it is not.

Declared pre-existing work

None of our own: Nirvana was started during the hackathon. Third-party parts used unchanged: Meterless H-MEM (vendored at 1742dec), the 103 TPC-DS texts from Apache Spark (3153f1e), DuckDB's dsdgen for the data, the Zetaris docker platform.

Limits

Results over 100,000 rows are truncated and marked. Rules cover four causes; anything else goes to the model or ends in keep. Nirvana reports verdicts and timings; it makes no dollar claim of its own.